

Grundlagenpraktikum: Rechnerarchitektur

Gruppe 125 – Abgabe zu Aufgabe A220

Wintersemester 2023/24

Markus-Ionut Vrancea

Kevin Kafexhi

Ota Takushima

1 Einleitung

Die digitale Bildverarbeitung behandelt grundlegende Methoden zur Bearbeitung und Aufbereitung von Bildern. Die Bildverarbeitungstechniken haben sich in den letzten zehn Jahren aufgrund zahlreicher Anwendungsbereiche, des wissenschaftlichen Interesses an diesem noch relativ jungen Forschungsgebiet und der zunehmenden Leistungsfähigkeit des Computers stark etabliert und verbessert. Themen wie Bildentrauschung, Segmentierung und Bildrekonstruktion sind in Bereichen wie der Nanotechnologie, der Sensorik und der medizinischen Bildgebung nicht mehr wegzudenken. Die Fernerkundung der Erde und des Universums, die aus militärischen Gründen motiviert wurde und ihre Bilder und Erkenntnisse in der Meteorologie, Ozeanographie und Astronomie anwendet, ist ein weiteres wichtiges Anwendungsgebiet dieser Technologien. Die digitale Bildverarbeitung ermöglicht es, die Informationen der Bilder aus diesen verschiedenen Aufnahmequellen zu bearbeiten und zu ändern. Sie ist daher ein wichtiger Bereich für weitere Forschung und Entwicklung.

1.1 Bildentrauschung

Eins der wichtigsten Verfahren in der Bildverarbeitung ist die Bildentrauschung. Was macht die Durchführung einer Bildentrauschung so wichtig? Unerwünschte Störungen können bei der Aufzeichnung und Übertragung von Bildern das Bild verfälschen und Informationen löschen. Der Versuch, die Störungen im Bild zu entfernen, beinhaltet nun die Wiederherstellung der Bildinformationen an den gestörten Stellen, indem die Informationen aus den umliegenden Pixeln verwendet werden. Nicht nur die Rauschentfernung, sondern auch die Kantenerhaltung, Erkennung und Extraktion sind bei der Modellierung eines effizienten Entrauschungsalgorithmus von entscheidender Bedeutung. Dass Bildkanten wichtige Informationsquellen sind, erklärt die Bedeutung dieser eng miteinander verbundenen Aufgaben bei der Verzerrung des Bildes. In Abbildung 1 wird veranschaulicht, wie die Entrauschung eines Bildes funktioniert.

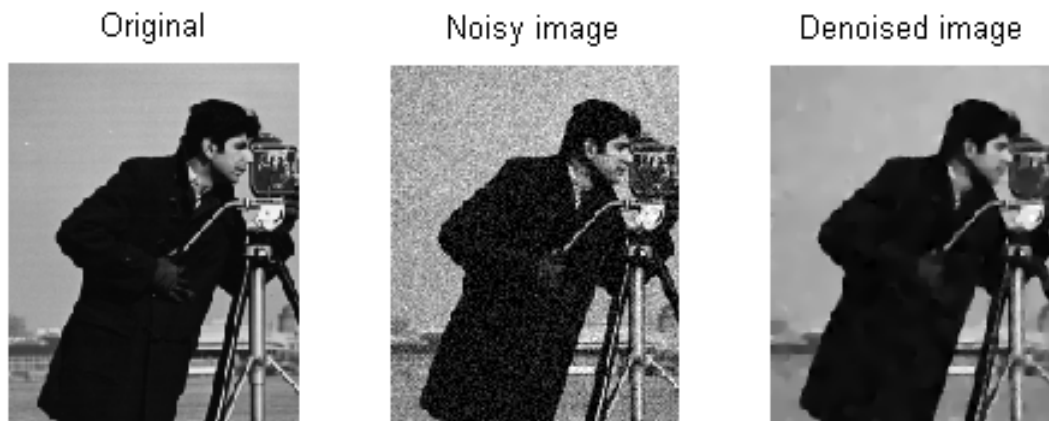


Abbildung 1: Ergebnis eines Bildes nach der Entrauschung [2]

1.2 Aufgabenstellung

Unsere Praktikumsaufgabe besteht darin eine effiziente Bearbeitungsweise zu finden, um ein farbiges Bild in Graustufen zu konvertieren und schließlich, mit Hilfe von dem Laplace-Filter und einer Weichbildung, das Bild zu entrauschen. Als Eingabe wird eine PPM Datei erwartet, die dem Algorithmus übergeben wird. Nach dem Ausführen des Algorithmus wird das entrauschte Bild im PGM Format in einer separaten Ausgabedatei gespeichert.

Unsere Lösung wird durch zwei verschiedene Implementierungen dargestellt. Die erste Implementierung ist die `denoise` Funktion, welche die Bildentrauschung Schritt für Schritt für jeden Pixel ausführt. Die zweite heißt `denoise_v1` und stellt eine effizientere Methode dar, den Algorithmus durchzuführen, indem SIMD-Befehle benutzt werden.

Zu diesem Zweck wird die Vorgehensweise des Algorithmus im theoretischen Teil analysiert und die Optimierung des Algorithmus im praktischen Teil beschrieben.

2 Lösungsansatz

Der folgende Abschnitt beschreibt den Ansatz unseres Programms, das eine PPM Datei erwartet und nach einer Sequenz von Verarbeitungen eine PGM Datei ausgibt.

2.1 Theoretischer Teil

Das PPM Bildformat (.ppm), das wir in diesem Projekt explizit als Eingabe behandeln werden, steht für „Portable Pixel Map“ und ist ein Teil der Netpbm, ein Bildformat das in 1988 veröffentlicht wurde [3]. Die Datei wird binär dargestellt und wird mit einem Header, das den Dateityp, die Breite, Höhe und die maximale Farbgröße des Bildes

enthält, gefolgt von den Pixeln des Bildes, wo jede 3 Bytes einem Pixelwert entspricht. Das Format kann auf Grund ihrer Einfachheit innerhalb einer E-Mail Nachricht ohne besondere Kapselung übertragen werden, sodass es möglichen Übersetzungen und Umkodierungen überleben kann und auf der anderen Seite auch leicht extrahierbar sein kann [1]. Da dieses Format die Bilddatei Pixel für Pixel speichert und Gruppen von Pixeln derselben Farben nicht zusammenfasst, kann dies, im Vergleich zu modernen Formaten wie JPEG oder PNG, eine relative große Datei sein.

Ein einfaches Beispiel des PPM Formats sieht wie folgt aus:

```
P6
# example from the man page
4 4
255
0 1 0 2 0 0 1 0 6 15 0 15
123 0 4 0 255 7 0 0 0 0 5 0
0 2 0 3 0 0 0 15 7 0 7 8
115 3 15 0 0 0 9 0 0 0 2 0
```

- P6: Eine „magische Zahl“ zur Identifizierung des Dateityps. Die magische Zahl eines PPM-Bildes sind die beiden Zeichen „P6“.
- #: Alle Zeichen von einem „#“ bis zum nächsten Zeilenumbruch sind ein Kommentar und werden ignoriert.
- 4 4: Die Breite und Höhe des Bildes in ASCII-Dezimalzahl, getrennt von einem Leerzeichen.
- 255: Der maximale Farbwert, wiederum in ASCII-Dezimalzahl. Für das Projekt ist er auf 255 festgelegt.
- Die unteren Nummern sind ein Raster von Zeilen in der Reihenfolge von oben nach unten. Jede Zeile besteht aus {Breite} Pixeln in der Reihenfolge von links nach rechts. Jedes Pixel ist ein Tripel von roten, grünen und blauen Mustern, in dieser Reihenfolge. Jedes Muster wird in reinem Binärformat durch 1 Byte dargestellt.

Eine 24bpp PPM Datei kann $2^{24} = 16777216$ Farben in RGB darstellen und jede Pixel lässt sich wie folgt darstellen:

$$p(x, y) = \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Das PGM Bildformat (.pgm), das wir in diesem Projekt als Ausgabe verwenden, steht für „Portable GrayMap“ und ist auch ein Teil der Netpbm. Dieses Format wird ebenfalls

binär dargestellt und behandelt die Schwarz-Weiße Version der obengenannten PPM. Das Format nutzt für ein Pixelwert nur 1 Byte. Da die Ausgabe unseres Programms in Graustufen erfolgt, und die Graustufendateien für jedes Pixel die gleichen RGB-Koeffizienten haben, ist es ineffizient, den gleichen Wert dreimal zu speichern, daher speichern wir nur einen davon. Dadurch wird auch die PGM-Ausgabedatei dreimal kleiner als die PPM. Deswegen haben wir uns entschieden, das PGM-Format als Ausgabe zu verwenden. Anstelle von „P6“, das eine PPM-Datei kennzeichnet, entsprechen die Buchstaben „P5“ der magischen Nummer einer PGM-Datei.

Graustufenkonvertierung

Um die Graustufenkonvertierung durchzuführen wenden wir die folgende Formel für jeden Pixel an:

$$D = \frac{a \cdot R + b \cdot G + c \cdot B}{a + b + c}$$

wobei die Konstanten a, b und c die folgenden Standardwerten haben:

$$a = 2.99, b = 5.87, c = 1.14$$

In dieser Formel wird die Gewichtung der rot, grün und blau Werte nach dem ITU-R BT.601 Standard Construction of Luminance [4] eingestellt. Dieser Standard wurde von der International Telecommunication Union für die digitale Videocodierung definiert und dient als eine effiziente Gewichtung für die Graustufenfilterung. Der berechnete Durchschnitt wird dann Pixel für Pixel in die neue PGM Datei eingefügt.

Laplace Filter

Das Graustufenbild wird dann einer Rauschreduzierung unterzogen, wofür wir zunächst den Laplace-Filter M^L auf das Graustufenbild Q anwenden, der zur Kantendetektion verwendet wird. Wir definieren Kanten als eine große Farbänderung zwischen benachbarten Pixeln. Er operiert durch Anwendung einer bestimmten Faltungsmatrix mit der Dimension 3 auf jedem Pixel des Bildes und rechnet sich wie folgt:

$$Q^L \approx M^L * Q \text{ mit } M^L = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

Diese Faltungsmatrix hat eine hohe, negative zentrale Pixelgewichtung und kleinere Gewichtungen für die umgebenden Pixel. Dadurch werden Unterschiede in der Pixelintensität betont, was zur Verstärkung von Kanten führt. Wichtig zu beachten ist, dass bei den Kanten des Bildes für Zugriff außerhalb des Definitionsbereichs der Funktion implizit den Wert 0 angenommen wird und deswegen die Kanten nach der Anwendung des Filters dunkler werden.

Weichzeichnung

Als nächsten Schritt berechnen wir mittels der folgenden Faltungsmatrix M^W die Weichzeichnung des Bildes:

$$Q^W = \frac{1}{16} \cdot M^W * Q \text{ mit } M^W = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

Die Weichzeichnung des Bildes wird durch Multiplikation des Wertes jedes Pixels und der umliegenden Pixeln mit den entsprechenden Werten in der Faltungsmatrix M^W erreicht. Die multiplizierten Werte werden dann aufaddiert und anschließend durch 16 geteilt. Hier wird das mittlere Pixel von den umgebenden Pixeln beeinflusst, wodurch das Pixel unscharf wird. Diese Methode wird auch an Kanten verwendet, mit der Annahme, dass Matrixmultiplikationen für Pixeln an Kanten und Seiten nicht vergütet werden, da sie nicht vorhanden sind. Die Summe der Pixelwerten an den Kanten werden durch 9, an den Seiten durch 12 und alle anderen durch 16 geteilt. Diese Zahlen ergeben sich aus der Differenz von 16 und der Summe der Zahlen, in der die entsprechenden Pixelwerte nicht vorhanden sind.

Berechnung des entrauschten Bildes

Das entrauschte Bild ergibt sich dann indem man das graugestufte Bild Q , das weichgezeichnete Bild Q^W und das Bild nach der Anwendung des Laplace-Filters Q^L wie folgt kombiniert:

$$Q'_{(x,y)} = \frac{|Q^L_{(x,y)}|}{1020} \cdot Q_{(x,y)} + \left(1 - \frac{|Q^L_{(x,y)}|}{1020}\right) \cdot Q^W_{(x,y)}$$

Um die Formel einfacher zu verstehen, teilen wir sie in zwei Teilen auf:

$$\frac{|Q^L_{(x,y)}|}{1020} \cdot Q_{(x,y)} \quad (1)$$

$$\left(1 - \frac{|Q^L_{(x,y)}|}{1020}\right) \cdot Q^W_{(x,y)} \quad (2)$$

(1): Dieser Teil besteht aus das Produkt von der absoluten Wert des Q^L und Q , dividiert mit 1020. Die Division durch 1020 normalisiert das Ergebnis des Laplace-Filters. Der Filter erzeugt Werte, die die Intensität der Kanten im Bild repräsentieren. Durch die Division wird dieser Wert in einen Faktor zwischen 0 und 1 umgewandelt und mit dem graugestuften Bild multipliziert, was an Stellen, an denen Q^L hohe Werte hat, die ursprünglichen Werte des Bildes behält, was auf scharfe Kanten hindeutet.

(2): Diese Operation bestimmt, wie viel vom weichgezeichneten Bild genutzt wird. An Stellen mit weniger Details oder Kanten wird mehr vom weichgezeichneten Bild genommen, was hilft, das Rauschen zu reduzieren.

Durch die Addition der beiden Seiten wird ein Bild ausgegeben, das bei den Kanten der Farben eines Bildes die Pixelwerte des originalen Bildes und sonst die weichgezeichneten Werten nimmt. Dies führt zu einem entrauschten Bild, bei dem die scharfen Kanten erhalten bleiben.

2.2 Praktischer Teil

In diesem Teil wird die Vorgehensweise der Implementierung einer Bildentrauschung beschrieben. Dafür werden zwei verschiedene implementierte Funktionen verwendet. Die Referenzimplementierung beschreibt einen effizienten Algorithmus für die Bildentrauschung, während die Hauptimplementierung die normale Durchführung des Algorithmus darstellt. Dabei wird auch das Rahmenprogramm beschrieben, durch welches der Nutzer das Programm mit verschiedenen Optionen starten kann.

Rahmenprogramm

Die Codebase besteht aus drei Quelldateien (.c) und zwei Headerdatei (.h). Im Header (`FileIO.h`) wird die Datenstruktur definiert, in welcher die Eingabedateien gespeichert werden. Die Eingabedatei ist durch ein struct repräsentiert, welcher die Dimensionen des Bilds, wie auch den maximalen Farbwert, gefolgt von einem Pointer, der auf die Werte des Pixels zeigt, speichert. Die Definition der Funktionen, die für die eigentliche Bildentrauschung benutzt werden, kann man in der `ImageDenoiser.h` Datei finden, während die einzelnen Implementierungen in der `ImageDenoiser.c` Datei gefunden werden können. In der `main.c` Datei ist die Main-Methode vorhanden, die das eigentliche Programm ausführt.

Die Command Line ermöglicht es dem Nutzer das Programm mit mehreren Parametern auszuführen. Dabei sind der Ausgabe Parameter (-o), gefolgt von dem entsprechenden Dateiname, und die Eingabe Datei, als Positional Argument, immer anzugeben. Die Eingabedatei sollte immer als letzter Parameter angegeben werden. Als Alternative kann sich der Nutzer mit dem Kommando „-h“ oder „-help“ auch eine Anweisung zur Nutzung der Parameter in der Konsole anzeigen lassen. Um die ausgeführte Version der Implementierung auszuwählen, muss der Parameter -v gefolgt von einer 0 (*Hauptimplementierung*) oder 1 (*Referenzimplementierung*) bestimmt werden. Falls dieser Parameter nicht angegeben wird, wird per default die Hauptimplementierung ausgeführt. Die Koeffizienten für die Graustufenkonvertierung können durch --coeffs Parameter angegeben werden. Falls dieser Parameter nicht vom Nutzer bestimmt sind, werden die default Werte $a = 2.99$, $b = 5.87$ und $c = 1.14$ verwendet. Um die Performanz der Methode zu testen, muss zusätzlich der -B Parameter gegeben werden. Nach dem Ausführen des Programms wird in diesem Fall die Laufzeit der Funktion in Sekunden in der Konsole angezeigt. Falls man die gesamte Laufzeit für mehrere Iterationen berechnen möchte, muss man dem -B Parameter zusätzlich noch eine positive Zahl, grösser als 0, hinzufügen. Diese Zahl gibt die Anzahl der durchzuführenden Iterationen an. Zusätzlich wurde der Parameter -k implementiert, der die Ausgabedateien beider Methoden vergleicht,

um die Korrektheit der Methoden zu testen. Ein Beispiel für die Ausführung des Codes wäre: `./denoise -V0 -B10 -o mandrill_out.pgm mandrill.ppm`

Implementierung

Als eine Programmiersprache ist C relativ hardwarenah und eignet sich daher hervorragend zum Schreiben effizienter Programme. Insbesondere nutzen wir für unser Bildenträuschungsprogramm Single Instruction, Multiple Data (SIMD), um mehrere Pixelwerte gleichzeitig zu verarbeiten. Diese Methode ist besonders vorteilhaft für Matrixoperationen, die einen Großteil unserer Entrauschungslogik ausmachen. Da für jede Matrixmultiplikation das Ergebnis der Berechnungen der Pixeln unabhängig voneinander sind, kann dies effektiv parallelisiert werden.

Unser Programm beinhaltet zwei Versionen: `denoise()` und `denoise_V1()`. Während `denoise()` eine sequenzielle Bearbeitung der Pixel für alle Methoden durchführt, nutzt `denoise_V1()` SIMD-Operationen für fast alle Schritte außer der letzten Berechnung. Die Graustufenkonvertierung in `denoise_V1()` verarbeitet vier Pixel gleichzeitig, und die Matrixmultiplikationen für die Laplace-Filter und die Weichzeichnung werden mit acht Pixeln gleichzeitig durchgeführt. Bei der Bearbeitung von Kanten und Ecken, den sogenannten Edge Cases, greift das Programm jedoch auf die Pixel-für-Pixel-Bearbeitung zurück.

3 Korrektheit

Die Korrektheit unseres Programms wird durch eine Äquivalenzprüfung nachgewiesen. In unserem Programm wird dies durch zwei verschiedene Implementierungen für dasselbe Problem erreicht.

Der erste Ansatz ist in der Methode `denoise` enthalten, die aus dem Ausgangsbild ein Graubild berechnet, darauf einen Laplace-Filter anwendet, eine Unschärfe für das Graubild berechnet und dann alle drei Ergebnisse kombiniert. Dieser Ansatz folgt der mathematischen Lösung aus der Problemdarstellung, die die Schritte, wie zuvor beschrieben, berechnet.

Die andere Lösung wird völlig anders implementiert. Anstatt eine einfache Berechnung zu verwenden, haben wir eine zweite Methode namens `denoise_V1`, die SIMD verwendet, erstellt.

Im Rahmenprogramm gibt es eine Option `-k`, die der Benutzer verwenden kann, um die Korrektheit beider Implementierungen zu überprüfen. Falls diese Option gesetzt ist, werden beide Methoden `denoise` und `denoise_V1` aufgerufen, die jeweiligen Ausgabewerte in verschiedenen Variablen `a` (für `denoise`) und `b` (für `denoise_V1`) gespeichert und letztendlich die beiden Ausgabedateien Byte für Byte verglichen. Falls die beiden Ausgabedateien gleich sind, gibt unser Programm in der Konsole: „They are the same picture!“ aus.

Wir haben die Ergebnisse für eine Vielzahl von Testfällen, die in der Datei `Implementierung/Tests` zu finden sind, getestet. Alle Tests waren erfolgreich, was bedeutet, dass

beide Implementierungen die selbe Ergebnisse für jeden Test erzeugt haben. Wenn die Ausgabe zweier verschiedener Ansätze, die dieselbe Datei als Eingabe enthalten, gleich ist, beweist das die Korrektheit beider Implementierungen.

4 Performanzanalyse

Für die Leistungsanalyse wurden die Tests auf einem Intel(R) Core(TM) i7-8550U CPU @ 1.80GHz 1.99 GHz RAM: 16 GB System durchgeführt. Die kleinste getestete Datei ist eine 64x64-Datei und die größte Datei ist 8192x8192 Pixel groß. Die Zwischendateien ergeben sich aus der Multiplikation der Größe der vorherigen Datei, mit 2. Die Dateien, die wir für die Durchführung der Tests verwendet haben, können in dem Implementierung/Tests Verzeichnis gefunden werden. Die Leistung jeder Version (denoise und denoise_V1) wurde sowohl mit dem Flag -O0, als auch mit dem Flag -O2 gemessen.

In Abbildung 2 kann man die Laufzeit der Tests, mit der Compileroption -O0, für beide Implementierungen, sehen. Für die kleinste getestete Datei ist die Leistung der SIMD-Version 1,4 Mal schneller als für die Basisversion. Bei der größten getesteten Datei ist die Berechnung mit der SIMD-Version 1,6 Mal schneller als mit der Standardversion. Dies liegt daran, dass SIMD bei der Verarbeitung größerer Dateien mehrere Komponenten parallel verarbeitet, was insgesamt zu einer schnelleren Verarbeitung führt als die Verarbeitung einzelner Elemente.

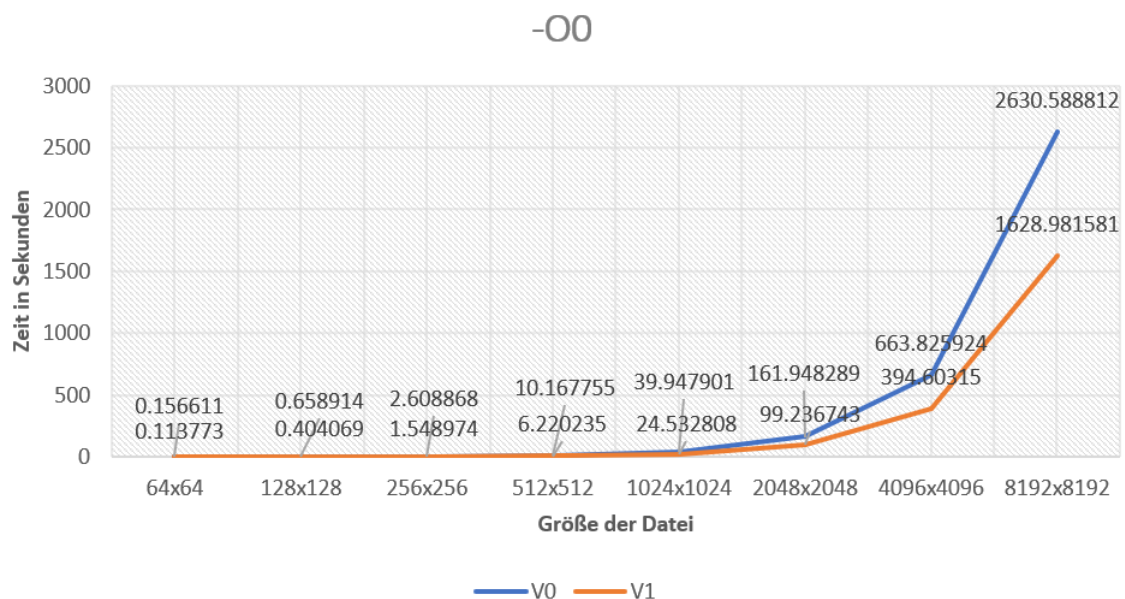


Abbildung 2: Performanz mit O0 [2]

In Abbildung 3 haben wir für die Laufzeitanalyse die gleichen Testdateien, aber dieses

Mal mit dem eingeschalteten Optimierungsflag -O2, verwendet. Anhand des Ergebnisses können wir bemerken, dass für die kleinste getestete Datei, die SIMD-Version 2 mal schneller berechnet wird als die normale Basisversion. Für die größte Datei ist die SIMD-Version 3,2 mal schneller. Der Grund dafür, dass die größeren Dateien mit SIMD schneller berechnet werden als mit der normalen Version, ist derselbe wie oben erwähnt. Wir können auch feststellen, dass die SIMD-Version mit dem -O2-Flag schneller als mit dem -O0-Flag ist. Das liegt daran, dass das -O2-Flag zusätzliche Optimierungen, wie Schleifenausrollung und Vektorisierung bietet, die der SIMD-Version zugute kommen. Es ist erwähnenswert, dass unsere SIMD-Version 3,5 Mal schneller als die Basisversion, für eine 1024x1024 große Datei, ist. Im Falle der 2048x2048-Datei ist sie dann 4,2 mal schneller. Die Analyse der Laufzeit beider Implementierungen beweist somit die Effizienz unserer SIMD-Version.

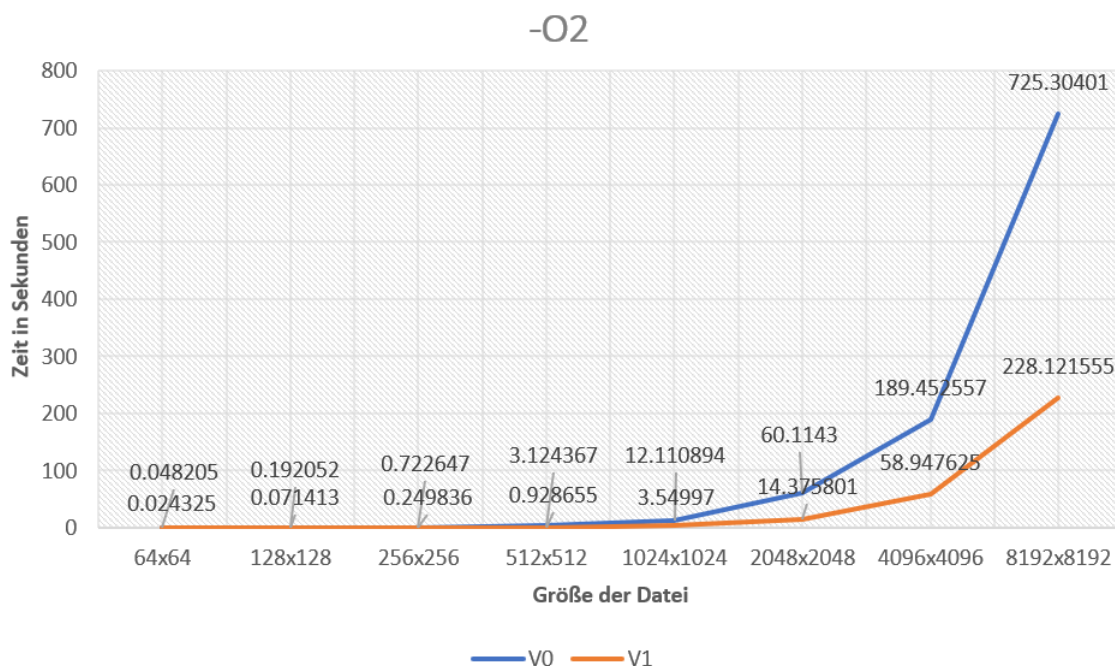


Abbildung 3: Performanz mit O2 [2]

5 Zusammenfassung und Ausblick

Zusammenfassend lässt sich sagen, dass das Ziel dieses Projekts darin besteht, eine Entrauschungsfunktion für die Entrauschung von Bildern bereitzustellen. Wir haben zunächst einen einfachen Algorithmus entwickelt, der auf die mathematischen Formeln der Übung basiert und jedes Bild Pixel für Pixel bearbeitet. Nach der Basisversion,

die wir zur Verfügung gestellt haben, dachten wir aufgrund der Tatsache, dass die Daten aus einer PPM-Datei aufeinanderfolgend sind, dass die beste Optimierungslösung, die Nutzung von SIMD Instruktionen sein würde. Nach diesem Erkenntnis haben wir auch eine zweite Version des Entrauschungsalgorithmus auf der Grundlage von SIMD-Anweisungen entwickelt, die effizienter als die erste ist. Nach der Durchführung von Laufzeitanalysen, für jede implementierte Version, kommen wir zu der Schlussfolgerung, dass unsere SIMD-Version in der Tat etwa drei Mal schneller ist als die Basisversion. Diese Ergebnisse tragen dazu bei, dass unsere Entscheidung, SIMD Instruktionen anzuwenden, eine richtige Entscheidung war.

Es ist aber noch zu erwähnen, dass unser Programm noch potenziell optimiert werden kann. Die Graustufenkonvertierung unseres Entrauschungsprogramms nimmt die RGB Werte eines Pixels und speichert sie in einem 128 bit Wert, wobei noch ein Padding Wert hinzugefügt wird. Diese Verschwendung kann durch das Einrücken des Rot-Wertes des nächsten Pixels und analog für die darauf folgenden Pixeln beseitigt werden, was wir aber auf Grund der Übersichtlichkeit des Programms nicht übernommen haben. Außerdem ist noch die Verwendung von Threads in den 3 Teilen der Entrauschungsfunktion für die Optimierung möglich.

Literatur

- [1] PPM Format Specification — netpbm.sourceforge.net. <https://netpbm.sourceforge.net/doc/ppm.html>. [Accessed 19-01-2024].
 - [2] Total variation denoising - Wikipedia — en.wikipedia.org. https://en.wikipedia.org/wiki/Total_variation_denoising. [Accessed 14-01-2024].
 - [3] Bryan Henderson. Netpbm History — netpbm.sourceforge.net. <https://netpbm.sourceforge.net/history.html>, 2023. [Accessed 19-01-2024].
 - [4] International Telecommunication Union. Recommendation itu-r bt.601-7. <https://www.itu.int/rec/R-REC-BT.601-7-201103-I/en>, 2011. Accessed: 2024-01-19.
-